PROJECT IDEAS

Portfolio Projects That Demonstrate Practical Testing Skills

100+ PAGE QA / AUTOMATION / SDET PROJECT PORTFOLIO MASTER GUIDE

CHOOSE → PLAN → TEST → AUTOMATE → INTEGRATE → DOCUMENT → DEMONSTRATE → DEFEND

A practical project-building guide for QA Engineers, Automation Engineers and SDETs. The goal is not to build another demo—it is to produce evidence of engineering ability.

Abhishek Kamal
QA Automation | SDET Career Guidance
Learn. Build. Get Hired.
abhishekkamal.dk+official@gmail.com

How to Use This Project Guide

Pick projects that let you demonstrate several layers of quality engineering instead of collecting many shallow repositories.

Portfolio evidence stack

Manual Testing → API → SQL/DB → UI Automation → Framework → CI/CD → Reports → Docker → Performance/Security → Documentation

Project selection rule

Project level	Recommended scope	Portfolio outcome
Starter	UI + manual + basic automation	Testing fundamentals
Intermediate	UI + API + DB + framework	Automation engineer evidence
Advanced	UI + API + DB + CI/CD + Docker	SDET evidence
Flagship	Distributed workflow + quality engineering	Senior-level discussion

01. E-Commerce Testing Platform

Product: Online shopping, catalog, cart, checkout and orders.

Recommended stack: UI testing, API testing, SQL validation, Playwright/Selenium, PyTest, CI/CD.

Project objective

Build a believable product workflow and demonstrate how a quality engineer would prevent, detect, diagnose and communicate failures.

Functional areas

• Authentication and authorization

• Core business workflow

• Negative and boundary cases

• Data validation

• Error handling

• Audit/history where relevant

Testing layers

Layer	What to demonstrate
Manual	Requirements, scenarios, risk-based cases, exploratory testing
API	Positive/negative, schema, auth, chaining, boundary conditions
Database	CRUD validation, joins, integrity, reconciliation
UI	Locators, synchronization, workflows, cross-browser
Automation	Fixtures, POM/component layer, data, assertions
CI/CD	Smoke gate, regression, reports, artifacts

Portfolio deliverables

■ README with architecture and setup

■ Test strategy and risk matrix

■ Test cases / scenarios

■ Automation repository

■ API collection or client

■ SQL validation examples

■ CI pipeline

■ Sample report and screenshots

■ Known limitations and future improvements

01. E-Commerce Testing Platform — Automation Blueprint

Recommended architecture: tests → workflows/page objects/API clients → fixtures/utilities → configuration/test data → reporting.

Suggested repository

tests/ UI/API/DB tests
pages/ or components/ UI abstraction
clients/ API clients
data/ controlled test data
utils/ reusable helpers
config/ environment settings
reports/ generated evidence
.github/ or Jenkinsfile CI pipeline

Definition of done

■ Tests run locally from a clean environment.

■ Tests can run in CI.

■ Failures provide useful evidence.

■ Test data does not create uncontrolled collisions.

■ README explains setup and architecture.

■ At least one meaningful quality metric is reported.

01. E-Commerce Testing Platform — Interview Defense Sheet

Questions an interviewer may ask

• Why did you choose this project?

• What were the highest-risk areas?

• What did you automate first and why?

• Why did you choose your framework?

• How did you manage test data?

• How did you handle flaky tests?

• How did you validate backend data?

• How does your CI pipeline work?

• What would you improve if this were production?

• What was the hardest problem you solved?

Your answer structure

Problem → Risk → Decision → Implementation → Evidence → Result → Trade-off → Next improvement

Portfolio score

Area	1	2	3	4	5
Testing depth	■	■	■	■	■
Automation quality	■	■	■	■	■
API/DB coverage	■	■	■	■	■
CI/CD	■	■	■	■	■
Documentation	■	■	■	■	■

02. Banking / FinTech Transaction System

Product: Accounts, transfers, beneficiaries, transaction history and limits.

Recommended stack: Risk-based testing, API automation, DB validation, security scenarios, audit validation.

Project objective

Build a believable product workflow and demonstrate how a quality engineer would prevent, detect, diagnose and communicate failures.

Functional areas

• Authentication and authorization

• Core business workflow

• Negative and boundary cases

• Data validation

• Error handling

• Audit/history where relevant

Testing layers

Layer	What to demonstrate
Manual	Requirements, scenarios, risk-based cases, exploratory testing
API	Positive/negative, schema, auth, chaining, boundary conditions
Database	CRUD validation, joins, integrity, reconciliation
UI	Locators, synchronization, workflows, cross-browser
Automation	Fixtures, POM/component layer, data, assertions
CI/CD	Smoke gate, regression, reports, artifacts

Portfolio deliverables

■ README with architecture and setup

■ Test strategy and risk matrix

■ Test cases / scenarios

■ Automation repository

■ API collection or client

■ SQL validation examples

■ CI pipeline

■ Sample report and screenshots

■ Known limitations and future improvements

02. Banking / FinTech Transaction System — Automation Blueprint

Recommended architecture: tests → workflows/page objects/API clients → fixtures/utilities → configuration/test data → reporting.

Suggested repository

tests/ UI/API/DB tests
pages/ or components/ UI abstraction
clients/ API clients
data/ controlled test data
utils/ reusable helpers
config/ environment settings
reports/ generated evidence
.github/ or Jenkinsfile CI pipeline

Definition of done

■ Tests run locally from a clean environment.

■ Tests can run in CI.

■ Failures provide useful evidence.

■ Test data does not create uncontrolled collisions.

■ README explains setup and architecture.

■ At least one meaningful quality metric is reported.

02. Banking / FinTech Transaction System — Interview Defense Sheet

Questions an interviewer may ask

• Why did you choose this project?

• What were the highest-risk areas?

• What did you automate first and why?

• Why did you choose your framework?

• How did you manage test data?

• How did you handle flaky tests?

• How did you validate backend data?

• How does your CI pipeline work?

• What would you improve if this were production?

• What was the hardest problem you solved?

Your answer structure

Problem → Risk → Decision → Implementation → Evidence → Result → Trade-off → Next improvement

Portfolio score

Area	1	2	3	4	5
Testing depth	■	■	■	■	■
Automation quality	■	■	■	■	■
API/DB coverage	■	■	■	■	■
CI/CD	■	■	■	■	■
Documentation	■	■	■	■	■

03. Travel Booking Platform

Product: Search, filters, booking, cancellation, payment and itinerary.

Recommended stack: Complex workflows, API chaining, date logic, UI automation, DB/API reconciliation.

Project objective

Build a believable product workflow and demonstrate how a quality engineer would prevent, detect, diagnose and communicate failures.

Functional areas

• Authentication and authorization

• Core business workflow

• Negative and boundary cases

• Data validation

• Error handling

• Audit/history where relevant

Testing layers

Layer	What to demonstrate
Manual	Requirements, scenarios, risk-based cases, exploratory testing
API	Positive/negative, schema, auth, chaining, boundary conditions
Database	CRUD validation, joins, integrity, reconciliation
UI	Locators, synchronization, workflows, cross-browser
Automation	Fixtures, POM/component layer, data, assertions
CI/CD	Smoke gate, regression, reports, artifacts

Portfolio deliverables

■ README with architecture and setup

■ Test strategy and risk matrix

■ Test cases / scenarios

■ Automation repository

■ API collection or client

■ SQL validation examples

■ CI pipeline

■ Sample report and screenshots

■ Known limitations and future improvements

03. Travel Booking Platform — Automation Blueprint

Recommended architecture: tests → workflows/page objects/API clients → fixtures/utilities → configuration/test data → reporting.

Suggested repository

tests/ UI/API/DB tests
pages/ or components/ UI abstraction
clients/ API clients
data/ controlled test data
utils/ reusable helpers
config/ environment settings
reports/ generated evidence
.github/ or Jenkinsfile CI pipeline

Definition of done

■ Tests run locally from a clean environment.

■ Tests can run in CI.

■ Failures provide useful evidence.

■ Test data does not create uncontrolled collisions.

■ README explains setup and architecture.

■ At least one meaningful quality metric is reported.

03. Travel Booking Platform — Interview Defense Sheet

Questions an interviewer may ask

• Why did you choose this project?

• What were the highest-risk areas?

• What did you automate first and why?

• Why did you choose your framework?

• How did you manage test data?

• How did you handle flaky tests?

• How did you validate backend data?

• How does your CI pipeline work?

• What would you improve if this were production?

• What was the hardest problem you solved?

Your answer structure

Problem → Risk → Decision → Implementation → Evidence → Result → Trade-off → Next improvement

Portfolio score

Area	1	2	3	4	5
Testing depth	■	■	■	■	■
Automation quality	■	■	■	■	■
API/DB coverage	■	■	■	■	■
CI/CD	■	■	■	■	■
Documentation	■	■	■	■	■

04. Food Delivery Platform

Product: Restaurant discovery, cart, coupons, delivery and order tracking.

Recommended stack: Location-like logic, API testing, async states, UI automation, notifications.

Project objective

Build a believable product workflow and demonstrate how a quality engineer would prevent, detect, diagnose and communicate failures.

Functional areas

• Authentication and authorization

• Core business workflow

• Negative and boundary cases

• Data validation

• Error handling

• Audit/history where relevant

Testing layers

Layer	What to demonstrate
Manual	Requirements, scenarios, risk-based cases, exploratory testing
API	Positive/negative, schema, auth, chaining, boundary conditions
Database	CRUD validation, joins, integrity, reconciliation
UI	Locators, synchronization, workflows, cross-browser
Automation	Fixtures, POM/component layer, data, assertions
CI/CD	Smoke gate, regression, reports, artifacts

Portfolio deliverables

■ README with architecture and setup

■ Test strategy and risk matrix

■ Test cases / scenarios

■ Automation repository

■ API collection or client

■ SQL validation examples

■ CI pipeline

■ Sample report and screenshots

■ Known limitations and future improvements

04. Food Delivery Platform — Automation Blueprint

Recommended architecture: tests → workflows/page objects/API clients → fixtures/utilities → configuration/test data → reporting.

Suggested repository

tests/ UI/API/DB tests
pages/ or components/ UI abstraction
clients/ API clients
data/ controlled test data
utils/ reusable helpers
config/ environment settings
reports/ generated evidence
.github/ or Jenkinsfile CI pipeline

Definition of done

■ Tests run locally from a clean environment.

■ Tests can run in CI.

■ Failures provide useful evidence.

■ Test data does not create uncontrolled collisions.

■ README explains setup and architecture.

■ At least one meaningful quality metric is reported.

04. Food Delivery Platform — Interview Defense Sheet

Questions an interviewer may ask

• Why did you choose this project?

• What were the highest-risk areas?

• What did you automate first and why?

• Why did you choose your framework?

• How did you manage test data?

• How did you handle flaky tests?

• How did you validate backend data?

• How does your CI pipeline work?

• What would you improve if this were production?

• What was the hardest problem you solved?

Your answer structure

Problem → Risk → Decision → Implementation → Evidence → Result → Trade-off → Next improvement

Portfolio score

Area	1	2	3	4	5
Testing depth	■	■	■	■	■
Automation quality	■	■	■	■	■
API/DB coverage	■	■	■	■	■
CI/CD	■	■	■	■	■
Documentation	■	■	■	■	■

05. HR / Employee Management System

Product: Employees, roles, leave, attendance and approvals.

Recommended stack: RBAC testing, CRUD APIs, SQL, workflow automation, audit trails.

Project objective

Build a believable product workflow and demonstrate how a quality engineer would prevent, detect, diagnose and communicate failures.

Functional areas

• Authentication and authorization

• Core business workflow

• Negative and boundary cases

• Data validation

• Error handling

• Audit/history where relevant

Testing layers

Layer	What to demonstrate
Manual	Requirements, scenarios, risk-based cases, exploratory testing
API	Positive/negative, schema, auth, chaining, boundary conditions
Database	CRUD validation, joins, integrity, reconciliation
UI	Locators, synchronization, workflows, cross-browser
Automation	Fixtures, POM/component layer, data, assertions
CI/CD	Smoke gate, regression, reports, artifacts

Portfolio deliverables

■ README with architecture and setup

■ Test strategy and risk matrix

■ Test cases / scenarios

■ Automation repository

■ API collection or client

■ SQL validation examples

■ CI pipeline

■ Sample report and screenshots

■ Known limitations and future improvements

05. HR / Employee Management System — Automation Blueprint

Recommended architecture: tests → workflows/page objects/API clients → fixtures/utilities → configuration/test data → reporting.

Suggested repository

tests/ UI/API/DB tests
pages/ or components/ UI abstraction
clients/ API clients
data/ controlled test data
utils/ reusable helpers
config/ environment settings
reports/ generated evidence
.github/ or Jenkinsfile CI pipeline

Definition of done

■ Tests run locally from a clean environment.

■ Tests can run in CI.

■ Failures provide useful evidence.

■ Test data does not create uncontrolled collisions.

■ README explains setup and architecture.

■ At least one meaningful quality metric is reported.

05. HR / Employee Management System — Interview Defense Sheet

Questions an interviewer may ask

• Why did you choose this project?

• What were the highest-risk areas?

• What did you automate first and why?

• Why did you choose your framework?

• How did you manage test data?

• How did you handle flaky tests?

• How did you validate backend data?

• How does your CI pipeline work?

• What would you improve if this were production?

• What was the hardest problem you solved?

Your answer structure

Problem → Risk → Decision → Implementation → Evidence → Result → Trade-off → Next improvement

Portfolio score

Area	1	2	3	4	5
Testing depth	■	■	■	■	■
Automation quality	■	■	■	■	■
API/DB coverage	■	■	■	■	■
CI/CD	■	■	■	■	■
Documentation	■	■	■	■	■

06. Insurance Claims Platform

Product: Policy, claim submission, documents, approval and settlement.

Recommended stack: Document workflows, business rules, API/DB testing, negative scenarios.

Project objective

Build a believable product workflow and demonstrate how a quality engineer would prevent, detect, diagnose and communicate failures.

Functional areas

• Authentication and authorization

• Core business workflow

• Negative and boundary cases

• Data validation

• Error handling

• Audit/history where relevant

Testing layers

Layer	What to demonstrate
Manual	Requirements, scenarios, risk-based cases, exploratory testing
API	Positive/negative, schema, auth, chaining, boundary conditions
Database	CRUD validation, joins, integrity, reconciliation
UI	Locators, synchronization, workflows, cross-browser
Automation	Fixtures, POM/component layer, data, assertions
CI/CD	Smoke gate, regression, reports, artifacts

Portfolio deliverables

■ README with architecture and setup

■ Test strategy and risk matrix

■ Test cases / scenarios

■ Automation repository

■ API collection or client

■ SQL validation examples

■ CI pipeline

■ Sample report and screenshots

■ Known limitations and future improvements

06. Insurance Claims Platform — Automation Blueprint

Recommended architecture: tests → workflows/page objects/API clients → fixtures/utilities → configuration/test data → reporting.

Suggested repository

tests/ UI/API/DB tests
pages/ or components/ UI abstraction
clients/ API clients
data/ controlled test data
utils/ reusable helpers
config/ environment settings
reports/ generated evidence
.github/ or Jenkinsfile CI pipeline

Definition of done

■ Tests run locally from a clean environment.

■ Tests can run in CI.

■ Failures provide useful evidence.

■ Test data does not create uncontrolled collisions.

■ README explains setup and architecture.

■ At least one meaningful quality metric is reported.

06. Insurance Claims Platform — Interview Defense Sheet

Questions an interviewer may ask

• Why did you choose this project?

• What were the highest-risk areas?

• What did you automate first and why?

• Why did you choose your framework?

• How did you manage test data?

• How did you handle flaky tests?

• How did you validate backend data?

• How does your CI pipeline work?

• What would you improve if this were production?

• What was the hardest problem you solved?

Your answer structure

Problem → Risk → Decision → Implementation → Evidence → Result → Trade-off → Next improvement

Portfolio score

Area	1	2	3	4	5
Testing depth	■	■	■	■	■
Automation quality	■	■	■	■	■
API/DB coverage	■	■	■	■	■
CI/CD	■	■	■	■	■
Documentation	■	■	■	■	■

07. Healthcare Appointment System

Product: Doctors, slots, patients, appointments and cancellations.

Recommended stack: Scheduling logic, concurrency scenarios, API, database and privacy-aware testing.

Project objective

Build a believable product workflow and demonstrate how a quality engineer would prevent, detect, diagnose and communicate failures.

Functional areas

• Authentication and authorization

• Core business workflow

• Negative and boundary cases

• Data validation

• Error handling

• Audit/history where relevant

Testing layers

Layer	What to demonstrate
Manual	Requirements, scenarios, risk-based cases, exploratory testing
API	Positive/negative, schema, auth, chaining, boundary conditions
Database	CRUD validation, joins, integrity, reconciliation
UI	Locators, synchronization, workflows, cross-browser
Automation	Fixtures, POM/component layer, data, assertions
CI/CD	Smoke gate, regression, reports, artifacts

Portfolio deliverables

■ README with architecture and setup

■ Test strategy and risk matrix

■ Test cases / scenarios

■ Automation repository

■ API collection or client

■ SQL validation examples

■ CI pipeline

■ Sample report and screenshots

■ Known limitations and future improvements

07. Healthcare Appointment System — Automation Blueprint

Recommended architecture: tests → workflows/page objects/API clients → fixtures/utilities → configuration/test data → reporting.

Suggested repository

tests/ UI/API/DB tests
pages/ or components/ UI abstraction
clients/ API clients
data/ controlled test data
utils/ reusable helpers
config/ environment settings
reports/ generated evidence
.github/ or Jenkinsfile CI pipeline

Definition of done

■ Tests run locally from a clean environment.

■ Tests can run in CI.

■ Failures provide useful evidence.

■ Test data does not create uncontrolled collisions.

■ README explains setup and architecture.

■ At least one meaningful quality metric is reported.

07. Healthcare Appointment System — Interview Defense Sheet

Questions an interviewer may ask

• Why did you choose this project?

• What were the highest-risk areas?

• What did you automate first and why?

• Why did you choose your framework?

• How did you manage test data?

• How did you handle flaky tests?

• How did you validate backend data?

• How does your CI pipeline work?

• What would you improve if this were production?

• What was the hardest problem you solved?

Your answer structure

Problem → Risk → Decision → Implementation → Evidence → Result → Trade-off → Next improvement

Portfolio score

Area	1	2	3	4	5
Testing depth	■	■	■	■	■
Automation quality	■	■	■	■	■
API/DB coverage	■	■	■	■	■
CI/CD	■	■	■	■	■
Documentation	■	■	■	■	■

08. Learning Management System

Product: Courses, enrollments, quizzes, progress and certificates.

Recommended stack: Role-based workflows, UI/API automation, data validation and reporting.

Project objective

Build a believable product workflow and demonstrate how a quality engineer would prevent, detect, diagnose and communicate failures.

Functional areas

• Authentication and authorization

• Core business workflow

• Negative and boundary cases

• Data validation

• Error handling

• Audit/history where relevant

Testing layers

Layer	What to demonstrate
Manual	Requirements, scenarios, risk-based cases, exploratory testing
API	Positive/negative, schema, auth, chaining, boundary conditions
Database	CRUD validation, joins, integrity, reconciliation
UI	Locators, synchronization, workflows, cross-browser
Automation	Fixtures, POM/component layer, data, assertions
CI/CD	Smoke gate, regression, reports, artifacts

Portfolio deliverables

■ README with architecture and setup

■ Test strategy and risk matrix

■ Test cases / scenarios

■ Automation repository

■ API collection or client

■ SQL validation examples

■ CI pipeline

■ Sample report and screenshots

■ Known limitations and future improvements

08. Learning Management System — Automation Blueprint

Recommended architecture: tests → workflows/page objects/API clients → fixtures/utilities → configuration/test data → reporting.

Suggested repository

tests/ UI/API/DB tests
pages/ or components/ UI abstraction
clients/ API clients
data/ controlled test data
utils/ reusable helpers
config/ environment settings
reports/ generated evidence
.github/ or Jenkinsfile CI pipeline

Definition of done

■ Tests run locally from a clean environment.

■ Tests can run in CI.

■ Failures provide useful evidence.

■ Test data does not create uncontrolled collisions.

■ README explains setup and architecture.

■ At least one meaningful quality metric is reported.

08. Learning Management System — Interview Defense Sheet

Questions an interviewer may ask

• Why did you choose this project?

• What were the highest-risk areas?

• What did you automate first and why?

• Why did you choose your framework?

• How did you manage test data?

• How did you handle flaky tests?

• How did you validate backend data?

• How does your CI pipeline work?

• What would you improve if this were production?

• What was the hardest problem you solved?

Your answer structure

Problem → Risk → Decision → Implementation → Evidence → Result → Trade-off → Next improvement

Portfolio score

Area	1	2	3	4	5
Testing depth	■	■	■	■	■
Automation quality	■	■	■	■	■
API/DB coverage	■	■	■	■	■
CI/CD	■	■	■	■	■
Documentation	■	■	■	■	■

09. SaaS Subscription Platform

Product: Plans, trials, upgrades, billing and cancellation.

Recommended stack: State transitions, payment-like workflows, API, DB, contract and regression testing.

Project objective

Build a believable product workflow and demonstrate how a quality engineer would prevent, detect, diagnose and communicate failures.

Functional areas

• Authentication and authorization

• Core business workflow

• Negative and boundary cases

• Data validation

• Error handling

• Audit/history where relevant

Testing layers

Layer	What to demonstrate
Manual	Requirements, scenarios, risk-based cases, exploratory testing
API	Positive/negative, schema, auth, chaining, boundary conditions
Database	CRUD validation, joins, integrity, reconciliation
UI	Locators, synchronization, workflows, cross-browser
Automation	Fixtures, POM/component layer, data, assertions
CI/CD	Smoke gate, regression, reports, artifacts

Portfolio deliverables

■ README with architecture and setup

■ Test strategy and risk matrix

■ Test cases / scenarios

■ Automation repository

■ API collection or client

■ SQL validation examples

■ CI pipeline

■ Sample report and screenshots

■ Known limitations and future improvements

09. SaaS Subscription Platform — Automation Blueprint

Recommended architecture: tests → workflows/page objects/API clients → fixtures/utilities → configuration/test data → reporting.

Suggested repository

tests/ UI/API/DB tests
pages/ or components/ UI abstraction
clients/ API clients
data/ controlled test data
utils/ reusable helpers
config/ environment settings
reports/ generated evidence
.github/ or Jenkinsfile CI pipeline

Definition of done

■ Tests run locally from a clean environment.

■ Tests can run in CI.

■ Failures provide useful evidence.

■ Test data does not create uncontrolled collisions.

■ README explains setup and architecture.

■ At least one meaningful quality metric is reported.

09. SaaS Subscription Platform — Interview Defense Sheet

Questions an interviewer may ask

• Why did you choose this project?

• What were the highest-risk areas?

• What did you automate first and why?

• Why did you choose your framework?

• How did you manage test data?

• How did you handle flaky tests?

• How did you validate backend data?

• How does your CI pipeline work?

• What would you improve if this were production?

• What was the hardest problem you solved?

Your answer structure

Problem → Risk → Decision → Implementation → Evidence → Result → Trade-off → Next improvement

Portfolio score

Area	1	2	3	4	5
Testing depth	■	■	■	■	■
Automation quality	■	■	■	■	■
API/DB coverage	■	■	■	■	■
CI/CD	■	■	■	■	■
Documentation	■	■	■	■	■

10. IoT / Smart Meter Platform

Product: Devices, registration, readings, alerts and remote commands.

Recommended stack: Telemetry validation, API/DB, asynchronous processing, logs and monitoring.

Project objective

Build a believable product workflow and demonstrate how a quality engineer would prevent, detect, diagnose and communicate failures.

Functional areas

• Authentication and authorization

• Core business workflow

• Negative and boundary cases

• Data validation

• Error handling

• Audit/history where relevant

Testing layers

Layer	What to demonstrate
Manual	Requirements, scenarios, risk-based cases, exploratory testing
API	Positive/negative, schema, auth, chaining, boundary conditions
Database	CRUD validation, joins, integrity, reconciliation
UI	Locators, synchronization, workflows, cross-browser
Automation	Fixtures, POM/component layer, data, assertions
CI/CD	Smoke gate, regression, reports, artifacts

Portfolio deliverables

■ README with architecture and setup

■ Test strategy and risk matrix

■ Test cases / scenarios

■ Automation repository

■ API collection or client

■ SQL validation examples

■ CI pipeline

■ Sample report and screenshots

■ Known limitations and future improvements

10. IoT / Smart Meter Platform — Automation Blueprint

Recommended architecture: tests → workflows/page objects/API clients → fixtures/utilities → configuration/test data → reporting.

Suggested repository

tests/ UI/API/DB tests
pages/ or components/ UI abstraction
clients/ API clients
data/ controlled test data
utils/ reusable helpers
config/ environment settings
reports/ generated evidence
.github/ or Jenkinsfile CI pipeline

Definition of done

■ Tests run locally from a clean environment.

■ Tests can run in CI.

■ Failures provide useful evidence.

■ Test data does not create uncontrolled collisions.

■ README explains setup and architecture.

■ At least one meaningful quality metric is reported.

10. IoT / Smart Meter Platform — Interview Defense Sheet

Questions an interviewer may ask

• Why did you choose this project?

• What were the highest-risk areas?

• What did you automate first and why?

• Why did you choose your framework?

• How did you manage test data?

• How did you handle flaky tests?

• How did you validate backend data?

• How does your CI pipeline work?

• What would you improve if this were production?

• What was the hardest problem you solved?

Your answer structure

Problem → Risk → Decision → Implementation → Evidence → Result → Trade-off → Next improvement

Portfolio score

Area	1	2	3	4	5
Testing depth	■	■	■	■	■
Automation quality	■	■	■	■	■
API/DB coverage	■	■	■	■	■
CI/CD	■	■	■	■	■
Documentation	■	■	■	■	■

11. Logistics / Shipment Platform

Product: Orders, warehouses, shipment status and delivery lifecycle.

Recommended stack: State-machine testing, API chaining, DB reconciliation and performance.

Project objective

Build a believable product workflow and demonstrate how a quality engineer would prevent, detect, diagnose and communicate failures.

Functional areas

• Authentication and authorization

• Core business workflow

• Negative and boundary cases

• Data validation

• Error handling

• Audit/history where relevant

Testing layers

Layer	What to demonstrate
Manual	Requirements, scenarios, risk-based cases, exploratory testing
API	Positive/negative, schema, auth, chaining, boundary conditions
Database	CRUD validation, joins, integrity, reconciliation
UI	Locators, synchronization, workflows, cross-browser
Automation	Fixtures, POM/component layer, data, assertions
CI/CD	Smoke gate, regression, reports, artifacts

Portfolio deliverables

■ README with architecture and setup

■ Test strategy and risk matrix

■ Test cases / scenarios

■ Automation repository

■ API collection or client

■ SQL validation examples

■ CI pipeline

■ Sample report and screenshots

■ Known limitations and future improvements

11. Logistics / Shipment Platform — Automation Blueprint

Recommended architecture: tests → workflows/page objects/API clients → fixtures/utilities → configuration/test data → reporting.

Suggested repository

tests/ UI/API/DB tests
pages/ or components/ UI abstraction
clients/ API clients
data/ controlled test data
utils/ reusable helpers
config/ environment settings
reports/ generated evidence
.github/ or Jenkinsfile CI pipeline

Definition of done

■ Tests run locally from a clean environment.

■ Tests can run in CI.

■ Failures provide useful evidence.

■ Test data does not create uncontrolled collisions.

■ README explains setup and architecture.

■ At least one meaningful quality metric is reported.

11. Logistics / Shipment Platform — Interview Defense Sheet

Questions an interviewer may ask

• Why did you choose this project?

• What were the highest-risk areas?

• What did you automate first and why?

• Why did you choose your framework?

• How did you manage test data?

• How did you handle flaky tests?

• How did you validate backend data?

• How does your CI pipeline work?

• What would you improve if this were production?

• What was the hardest problem you solved?

Your answer structure

Problem → Risk → Decision → Implementation → Evidence → Result → Trade-off → Next improvement

Portfolio score

Area	1	2	3	4	5
Testing depth	■	■	■	■	■
Automation quality	■	■	■	■	■
API/DB coverage	■	■	■	■	■
CI/CD	■	■	■	■	■
Documentation	■	■	■	■	■

12. CRM / Sales Pipeline

Product: Leads, accounts, opportunities, activities and dashboards.

Recommended stack: CRUD, permissions, workflow, API, UI and data-quality testing.

Project objective

Build a believable product workflow and demonstrate how a quality engineer would prevent, detect, diagnose and communicate failures.

Functional areas

• Authentication and authorization

• Core business workflow

• Negative and boundary cases

• Data validation

• Error handling

• Audit/history where relevant

Testing layers

Layer	What to demonstrate
Manual	Requirements, scenarios, risk-based cases, exploratory testing
API	Positive/negative, schema, auth, chaining, boundary conditions
Database	CRUD validation, joins, integrity, reconciliation
UI	Locators, synchronization, workflows, cross-browser
Automation	Fixtures, POM/component layer, data, assertions
CI/CD	Smoke gate, regression, reports, artifacts

Portfolio deliverables

■ README with architecture and setup

■ Test strategy and risk matrix

■ Test cases / scenarios

■ Automation repository

■ API collection or client

■ SQL validation examples

■ CI pipeline

■ Sample report and screenshots

■ Known limitations and future improvements

12. CRM / Sales Pipeline — Automation Blueprint

Recommended architecture: tests → workflows/page objects/API clients → fixtures/utilities → configuration/test data → reporting.

Suggested repository

tests/ UI/API/DB tests
pages/ or components/ UI abstraction
clients/ API clients
data/ controlled test data
utils/ reusable helpers
config/ environment settings
reports/ generated evidence
.github/ or Jenkinsfile CI pipeline

Definition of done

■ Tests run locally from a clean environment.

■ Tests can run in CI.

■ Failures provide useful evidence.

■ Test data does not create uncontrolled collisions.

■ README explains setup and architecture.

■ At least one meaningful quality metric is reported.

12. CRM / Sales Pipeline — Interview Defense Sheet

Questions an interviewer may ask

• Why did you choose this project?

• What were the highest-risk areas?

• What did you automate first and why?

• Why did you choose your framework?

• How did you manage test data?

• How did you handle flaky tests?

• How did you validate backend data?

• How does your CI pipeline work?

• What would you improve if this were production?

• What was the hardest problem you solved?

Your answer structure

Problem → Risk → Decision → Implementation → Evidence → Result → Trade-off → Next improvement

Portfolio score

Area	1	2	3	4	5
Testing depth	■	■	■	■	■
Automation quality	■	■	■	■	■
API/DB coverage	■	■	■	■	■
CI/CD	■	■	■	■	■
Documentation	■	■	■	■	■

13. Job Portal

Product: Profiles, resumes, jobs, applications and recruiter workflows.

Recommended stack: Search/filter testing, file upload, API, DB and role-based access.

Project objective

Build a believable product workflow and demonstrate how a quality engineer would prevent, detect, diagnose and communicate failures.

Functional areas

• Authentication and authorization

• Core business workflow

• Negative and boundary cases

• Data validation

• Error handling

• Audit/history where relevant

Testing layers

Layer	What to demonstrate
Manual	Requirements, scenarios, risk-based cases, exploratory testing
API	Positive/negative, schema, auth, chaining, boundary conditions
Database	CRUD validation, joins, integrity, reconciliation
UI	Locators, synchronization, workflows, cross-browser
Automation	Fixtures, POM/component layer, data, assertions
CI/CD	Smoke gate, regression, reports, artifacts

Portfolio deliverables

■ README with architecture and setup

■ Test strategy and risk matrix

■ Test cases / scenarios

■ Automation repository

■ API collection or client

■ SQL validation examples

■ CI pipeline

■ Sample report and screenshots

■ Known limitations and future improvements

13. Job Portal — Automation Blueprint

Recommended architecture: tests → workflows/page objects/API clients → fixtures/utilities → configuration/test data → reporting.

Suggested repository

tests/ UI/API/DB tests
pages/ or components/ UI abstraction
clients/ API clients
data/ controlled test data
utils/ reusable helpers
config/ environment settings
reports/ generated evidence
.github/ or Jenkinsfile CI pipeline

Definition of done

■ Tests run locally from a clean environment.

■ Tests can run in CI.

■ Failures provide useful evidence.

■ Test data does not create uncontrolled collisions.

■ README explains setup and architecture.

■ At least one meaningful quality metric is reported.

13. Job Portal — Interview Defense Sheet

Questions an interviewer may ask

• Why did you choose this project?

• What were the highest-risk areas?

• What did you automate first and why?

• Why did you choose your framework?

• How did you manage test data?

• How did you handle flaky tests?

• How did you validate backend data?

• How does your CI pipeline work?

• What would you improve if this were production?

• What was the hardest problem you solved?

Your answer structure

Problem → Risk → Decision → Implementation → Evidence → Result → Trade-off → Next improvement

Portfolio score

Area	1	2	3	4	5
Testing depth	■	■	■	■	■
Automation quality	■	■	■	■	■
API/DB coverage	■	■	■	■	■
CI/CD	■	■	■	■	■
Documentation	■	■	■	■	■

14. Digital Wallet

Product: Wallet balance, transfers, recharge and transaction history.

Recommended stack: Financial rules, idempotency, concurrency, API/DB and security testing.

Project objective

Build a believable product workflow and demonstrate how a quality engineer would prevent, detect, diagnose and communicate failures.

Functional areas

• Authentication and authorization

• Core business workflow

• Negative and boundary cases

• Data validation

• Error handling

• Audit/history where relevant

Testing layers

Layer	What to demonstrate
Manual	Requirements, scenarios, risk-based cases, exploratory testing
API	Positive/negative, schema, auth, chaining, boundary conditions
Database	CRUD validation, joins, integrity, reconciliation
UI	Locators, synchronization, workflows, cross-browser
Automation	Fixtures, POM/component layer, data, assertions
CI/CD	Smoke gate, regression, reports, artifacts

Portfolio deliverables

■ README with architecture and setup

■ Test strategy and risk matrix

■ Test cases / scenarios

■ Automation repository

■ API collection or client

■ SQL validation examples

■ CI pipeline

■ Sample report and screenshots

■ Known limitations and future improvements

14. Digital Wallet — Automation Blueprint

Recommended architecture: tests → workflows/page objects/API clients → fixtures/utilities → configuration/test data → reporting.

Suggested repository

tests/ UI/API/DB tests
pages/ or components/ UI abstraction
clients/ API clients
data/ controlled test data
utils/ reusable helpers
config/ environment settings
reports/ generated evidence
.github/ or Jenkinsfile CI pipeline

Definition of done

■ Tests run locally from a clean environment.

■ Tests can run in CI.

■ Failures provide useful evidence.

■ Test data does not create uncontrolled collisions.

■ README explains setup and architecture.

■ At least one meaningful quality metric is reported.

14. Digital Wallet — Interview Defense Sheet

Questions an interviewer may ask

• Why did you choose this project?

• What were the highest-risk areas?

• What did you automate first and why?

• Why did you choose your framework?

• How did you manage test data?

• How did you handle flaky tests?

• How did you validate backend data?

• How does your CI pipeline work?

• What would you improve if this were production?

• What was the hardest problem you solved?

Your answer structure

Problem → Risk → Decision → Implementation → Evidence → Result → Trade-off → Next improvement

Portfolio score

Area	1	2	3	4	5
Testing depth	■	■	■	■	■
Automation quality	■	■	■	■	■
API/DB coverage	■	■	■	■	■
CI/CD	■	■	■	■	■
Documentation	■	■	■	■	■

15. Social Media Mini Platform

Product: Profiles, posts, comments, likes, follows and moderation.

Recommended stack: API-heavy testing, abuse cases, pagination, concurrency and UI automation.

Project objective

Build a believable product workflow and demonstrate how a quality engineer would prevent, detect, diagnose and communicate failures.

Functional areas

• Authentication and authorization

• Core business workflow

• Negative and boundary cases

• Data validation

• Error handling

• Audit/history where relevant

Testing layers

Layer	What to demonstrate
Manual	Requirements, scenarios, risk-based cases, exploratory testing
API	Positive/negative, schema, auth, chaining, boundary conditions
Database	CRUD validation, joins, integrity, reconciliation
UI	Locators, synchronization, workflows, cross-browser
Automation	Fixtures, POM/component layer, data, assertions
CI/CD	Smoke gate, regression, reports, artifacts

Portfolio deliverables

■ README with architecture and setup

■ Test strategy and risk matrix

■ Test cases / scenarios

■ Automation repository

■ API collection or client

■ SQL validation examples

■ CI pipeline

■ Sample report and screenshots

■ Known limitations and future improvements

15. Social Media Mini Platform — Automation Blueprint

Recommended architecture: tests → workflows/page objects/API clients → fixtures/utilities → configuration/test data → reporting.

Suggested repository

tests/ UI/API/DB tests
pages/ or components/ UI abstraction
clients/ API clients
data/ controlled test data
utils/ reusable helpers
config/ environment settings
reports/ generated evidence
.github/ or Jenkinsfile CI pipeline

Definition of done

■ Tests run locally from a clean environment.

■ Tests can run in CI.

■ Failures provide useful evidence.

■ Test data does not create uncontrolled collisions.

■ README explains setup and architecture.

■ At least one meaningful quality metric is reported.

15. Social Media Mini Platform — Interview Defense Sheet

Questions an interviewer may ask

• Why did you choose this project?

• What were the highest-risk areas?

• What did you automate first and why?

• Why did you choose your framework?

• How did you manage test data?

• How did you handle flaky tests?

• How did you validate backend data?

• How does your CI pipeline work?

• What would you improve if this were production?

• What was the hardest problem you solved?

Your answer structure

Problem → Risk → Decision → Implementation → Evidence → Result → Trade-off → Next improvement

Portfolio score

Area	1	2	3	4	5
Testing depth	■	■	■	■	■
Automation quality	■	■	■	■	■
API/DB coverage	■	■	■	■	■
CI/CD	■	■	■	■	■
Documentation	■	■	■	■	■

16. Inventory Management System

Product: Products, stock, purchase, sales and reorder thresholds.

Recommended stack: Data integrity, business rules, API/DB, reports and CI.

Project objective

Build a believable product workflow and demonstrate how a quality engineer would prevent, detect, diagnose and communicate failures.

Functional areas

• Authentication and authorization

• Core business workflow

• Negative and boundary cases

• Data validation

• Error handling

• Audit/history where relevant

Testing layers

Layer	What to demonstrate
Manual	Requirements, scenarios, risk-based cases, exploratory testing
API	Positive/negative, schema, auth, chaining, boundary conditions
Database	CRUD validation, joins, integrity, reconciliation
UI	Locators, synchronization, workflows, cross-browser
Automation	Fixtures, POM/component layer, data, assertions
CI/CD	Smoke gate, regression, reports, artifacts

Portfolio deliverables

■ README with architecture and setup

■ Test strategy and risk matrix

■ Test cases / scenarios

■ Automation repository

■ API collection or client

■ SQL validation examples

■ CI pipeline

■ Sample report and screenshots

■ Known limitations and future improvements

16. Inventory Management System — Automation Blueprint

Recommended architecture: tests → workflows/page objects/API clients → fixtures/utilities → configuration/test data → reporting.

Suggested repository

tests/ UI/API/DB tests
pages/ or components/ UI abstraction
clients/ API clients
data/ controlled test data
utils/ reusable helpers
config/ environment settings
reports/ generated evidence
.github/ or Jenkinsfile CI pipeline

Definition of done

■ Tests run locally from a clean environment.

■ Tests can run in CI.

■ Failures provide useful evidence.

■ Test data does not create uncontrolled collisions.

■ README explains setup and architecture.

■ At least one meaningful quality metric is reported.

16. Inventory Management System — Interview Defense Sheet

Questions an interviewer may ask

• Why did you choose this project?

• What were the highest-risk areas?

• What did you automate first and why?

• Why did you choose your framework?

• How did you manage test data?

• How did you handle flaky tests?

• How did you validate backend data?

• How does your CI pipeline work?

• What would you improve if this were production?

• What was the hardest problem you solved?

Your answer structure

Problem → Risk → Decision → Implementation → Evidence → Result → Trade-off → Next improvement

Portfolio score

Area	1	2	3	4	5
Testing depth	■	■	■	■	■
Automation quality	■	■	■	■	■
API/DB coverage	■	■	■	■	■
CI/CD	■	■	■	■	■
Documentation	■	■	■	■	■

17. Event Ticketing Platform

Product: Events, seats, booking, cancellation and ticket generation.

Recommended stack: Concurrency, seat locking, payment-like flow, API and end-to-end automation.

Project objective

Build a believable product workflow and demonstrate how a quality engineer would prevent, detect, diagnose and communicate failures.

Functional areas

• Authentication and authorization

• Core business workflow

• Negative and boundary cases

• Data validation

• Error handling

• Audit/history where relevant

Testing layers

Layer	What to demonstrate
Manual	Requirements, scenarios, risk-based cases, exploratory testing
API	Positive/negative, schema, auth, chaining, boundary conditions
Database	CRUD validation, joins, integrity, reconciliation
UI	Locators, synchronization, workflows, cross-browser
Automation	Fixtures, POM/component layer, data, assertions
CI/CD	Smoke gate, regression, reports, artifacts

Portfolio deliverables

■ README with architecture and setup

■ Test strategy and risk matrix

■ Test cases / scenarios

■ Automation repository

■ API collection or client

■ SQL validation examples

■ CI pipeline

■ Sample report and screenshots

■ Known limitations and future improvements

17. Event Ticketing Platform — Automation Blueprint

Recommended architecture: tests → workflows/page objects/API clients → fixtures/utilities → configuration/test data → reporting.

Suggested repository

tests/ UI/API/DB tests
pages/ or components/ UI abstraction
clients/ API clients
data/ controlled test data
utils/ reusable helpers
config/ environment settings
reports/ generated evidence
.github/ or Jenkinsfile CI pipeline

Definition of done

■ Tests run locally from a clean environment.

■ Tests can run in CI.

■ Failures provide useful evidence.

■ Test data does not create uncontrolled collisions.

■ README explains setup and architecture.

■ At least one meaningful quality metric is reported.

17. Event Ticketing Platform — Interview Defense Sheet

Questions an interviewer may ask

• Why did you choose this project?

• What were the highest-risk areas?

• What did you automate first and why?

• Why did you choose your framework?

• How did you manage test data?

• How did you handle flaky tests?

• How did you validate backend data?

• How does your CI pipeline work?

• What would you improve if this were production?

• What was the hardest problem you solved?

Your answer structure

Problem → Risk → Decision → Implementation → Evidence → Result → Trade-off → Next improvement

Portfolio score

Area	1	2	3	4	5
Testing depth	■	■	■	■	■
Automation quality	■	■	■	■	■
API/DB coverage	■	■	■	■	■
CI/CD	■	■	■	■	■
Documentation	■	■	■	■	■

18. Ride Booking Platform

Product: Rider, driver, trip lifecycle, fare and cancellation.

Recommended stack: State transitions, pricing rules, API, asynchronous updates and reliability.

Project objective

Build a believable product workflow and demonstrate how a quality engineer would prevent, detect, diagnose and communicate failures.

Functional areas

• Authentication and authorization

• Core business workflow

• Negative and boundary cases

• Data validation

• Error handling

• Audit/history where relevant

Testing layers

Layer	What to demonstrate
Manual	Requirements, scenarios, risk-based cases, exploratory testing
API	Positive/negative, schema, auth, chaining, boundary conditions
Database	CRUD validation, joins, integrity, reconciliation
UI	Locators, synchronization, workflows, cross-browser
Automation	Fixtures, POM/component layer, data, assertions
CI/CD	Smoke gate, regression, reports, artifacts

Portfolio deliverables

■ README with architecture and setup

■ Test strategy and risk matrix

■ Test cases / scenarios

■ Automation repository

■ API collection or client

■ SQL validation examples

■ CI pipeline

■ Sample report and screenshots

■ Known limitations and future improvements

18. Ride Booking Platform — Automation Blueprint

Recommended architecture: tests → workflows/page objects/API clients → fixtures/utilities → configuration/test data → reporting.

Suggested repository

tests/ UI/API/DB tests
pages/ or components/ UI abstraction
clients/ API clients
data/ controlled test data
utils/ reusable helpers
config/ environment settings
reports/ generated evidence
.github/ or Jenkinsfile CI pipeline

Definition of done

■ Tests run locally from a clean environment.

■ Tests can run in CI.

■ Failures provide useful evidence.

■ Test data does not create uncontrolled collisions.

■ README explains setup and architecture.

■ At least one meaningful quality metric is reported.

18. Ride Booking Platform — Interview Defense Sheet

Questions an interviewer may ask

• Why did you choose this project?

• What were the highest-risk areas?

• What did you automate first and why?

• Why did you choose your framework?

• How did you manage test data?

• How did you handle flaky tests?

• How did you validate backend data?

• How does your CI pipeline work?

• What would you improve if this were production?

• What was the hardest problem you solved?

Your answer structure

Problem → Risk → Decision → Implementation → Evidence → Result → Trade-off → Next improvement

Portfolio score

Area	1	2	3	4	5
Testing depth	■	■	■	■	■
Automation quality	■	■	■	■	■
API/DB coverage	■	■	■	■	■
CI/CD	■	■	■	■	■
Documentation	■	■	■	■	■

19. API-Only Microservices Demo

Product: Several services communicating through REST APIs.

Recommended stack: Contract testing, service virtualization concepts, API automation, Docker and CI.

Project objective

Build a believable product workflow and demonstrate how a quality engineer would prevent, detect, diagnose and communicate failures.

Functional areas

• Authentication and authorization

• Core business workflow

• Negative and boundary cases

• Data validation

• Error handling

• Audit/history where relevant

Testing layers

Layer	What to demonstrate
Manual	Requirements, scenarios, risk-based cases, exploratory testing
API	Positive/negative, schema, auth, chaining, boundary conditions
Database	CRUD validation, joins, integrity, reconciliation
UI	Locators, synchronization, workflows, cross-browser
Automation	Fixtures, POM/component layer, data, assertions
CI/CD	Smoke gate, regression, reports, artifacts

Portfolio deliverables

■ README with architecture and setup

■ Test strategy and risk matrix

■ Test cases / scenarios

■ Automation repository

■ API collection or client

■ SQL validation examples

■ CI pipeline

■ Sample report and screenshots

■ Known limitations and future improvements

19. API-Only Microservices Demo — Automation Blueprint

Recommended architecture: tests → workflows/page objects/API clients → fixtures/utilities → configuration/test data → reporting.

Suggested repository

tests/ UI/API/DB tests
pages/ or components/ UI abstraction
clients/ API clients
data/ controlled test data
utils/ reusable helpers
config/ environment settings
reports/ generated evidence
.github/ or Jenkinsfile CI pipeline

Definition of done

■ Tests run locally from a clean environment.

■ Tests can run in CI.

■ Failures provide useful evidence.

■ Test data does not create uncontrolled collisions.

■ README explains setup and architecture.

■ At least one meaningful quality metric is reported.

19. API-Only Microservices Demo — Interview Defense Sheet

Questions an interviewer may ask

• Why did you choose this project?

• What were the highest-risk areas?

• What did you automate first and why?

• Why did you choose your framework?

• How did you manage test data?

• How did you handle flaky tests?

• How did you validate backend data?

• How does your CI pipeline work?

• What would you improve if this were production?

• What was the hardest problem you solved?

Your answer structure

Problem → Risk → Decision → Implementation → Evidence → Result → Trade-off → Next improvement

Portfolio score

Area	1	2	3	4	5
Testing depth	■	■	■	■	■
Automation quality	■	■	■	■	■
API/DB coverage	■	■	■	■	■
CI/CD	■	■	■	■	■
Documentation	■	■	■	■	■

20. AI / RAG Assistant

Product: Document ingestion, retrieval, answer generation and citations.

Recommended stack: LLM evaluation, groundedness, retrieval tests, prompt regression and security.

Project objective

Build a believable product workflow and demonstrate how a quality engineer would prevent, detect, diagnose and communicate failures.

Functional areas

• Authentication and authorization

• Core business workflow

• Negative and boundary cases

• Data validation

• Error handling

• Audit/history where relevant

Testing layers

Layer	What to demonstrate
Manual	Requirements, scenarios, risk-based cases, exploratory testing
API	Positive/negative, schema, auth, chaining, boundary conditions
Database	CRUD validation, joins, integrity, reconciliation
UI	Locators, synchronization, workflows, cross-browser
Automation	Fixtures, POM/component layer, data, assertions
CI/CD	Smoke gate, regression, reports, artifacts

Portfolio deliverables

■ README with architecture and setup

■ Test strategy and risk matrix

■ Test cases / scenarios

■ Automation repository

■ API collection or client

■ SQL validation examples

■ CI pipeline

■ Sample report and screenshots

■ Known limitations and future improvements

20. AI / RAG Assistant — Automation Blueprint

Recommended architecture: tests → workflows/page objects/API clients → fixtures/utilities → configuration/test data → reporting.

Suggested repository

tests/ UI/API/DB tests
pages/ or components/ UI abstraction
clients/ API clients
data/ controlled test data
utils/ reusable helpers
config/ environment settings
reports/ generated evidence
.github/ or Jenkinsfile CI pipeline

Definition of done

■ Tests run locally from a clean environment.

■ Tests can run in CI.

■ Failures provide useful evidence.

■ Test data does not create uncontrolled collisions.

■ README explains setup and architecture.

■ At least one meaningful quality metric is reported.

20. AI / RAG Assistant — Interview Defense Sheet

Questions an interviewer may ask

• Why did you choose this project?

• What were the highest-risk areas?

• What did you automate first and why?

• Why did you choose your framework?

• How did you manage test data?

• How did you handle flaky tests?

• How did you validate backend data?

• How does your CI pipeline work?

• What would you improve if this were production?

• What was the hardest problem you solved?

Your answer structure

Problem → Risk → Decision → Implementation → Evidence → Result → Trade-off → Next improvement

Portfolio score

Area	1	2	3	4	5
Testing depth	■	■	■	■	■
Automation quality	■	■	■	■	■
API/DB coverage	■	■	■	■	■
CI/CD	■	■	■	■	■
Documentation	■	■	■	■	■

21. Performance Engineering Lab

Product: A realistic API/service workload with measurable SLAs.

Recommended stack: JMeter, workload modelling, p95/p99, bottleneck analysis and Grafana.

Project objective

Build a believable product workflow and demonstrate how a quality engineer would prevent, detect, diagnose and communicate failures.

Functional areas

• Authentication and authorization

• Core business workflow

• Negative and boundary cases

• Data validation

• Error handling

• Audit/history where relevant

Testing layers

Layer	What to demonstrate
Manual	Requirements, scenarios, risk-based cases, exploratory testing
API	Positive/negative, schema, auth, chaining, boundary conditions
Database	CRUD validation, joins, integrity, reconciliation
UI	Locators, synchronization, workflows, cross-browser
Automation	Fixtures, POM/component layer, data, assertions
CI/CD	Smoke gate, regression, reports, artifacts

Portfolio deliverables

■ README with architecture and setup

■ Test strategy and risk matrix

■ Test cases / scenarios

■ Automation repository

■ API collection or client

■ SQL validation examples

■ CI pipeline

■ Sample report and screenshots

■ Known limitations and future improvements

21. Performance Engineering Lab — Automation Blueprint

Recommended architecture: tests → workflows/page objects/API clients → fixtures/utilities → configuration/test data → reporting.

Suggested repository

tests/ UI/API/DB tests
pages/ or components/ UI abstraction
clients/ API clients
data/ controlled test data
utils/ reusable helpers
config/ environment settings
reports/ generated evidence
.github/ or Jenkinsfile CI pipeline

Definition of done

■ Tests run locally from a clean environment.

■ Tests can run in CI.

■ Failures provide useful evidence.

■ Test data does not create uncontrolled collisions.

■ README explains setup and architecture.

■ At least one meaningful quality metric is reported.

21. Performance Engineering Lab — Interview Defense Sheet

Questions an interviewer may ask

• Why did you choose this project?

• What were the highest-risk areas?

• What did you automate first and why?

• Why did you choose your framework?

• How did you manage test data?

• How did you handle flaky tests?

• How did you validate backend data?

• How does your CI pipeline work?

• What would you improve if this were production?

• What was the hardest problem you solved?

Your answer structure

Problem → Risk → Decision → Implementation → Evidence → Result → Trade-off → Next improvement

Portfolio score

Area	1	2	3	4	5
Testing depth	■	■	■	■	■
Automation quality	■	■	■	■	■
API/DB coverage	■	■	■	■	■
CI/CD	■	■	■	■	■
Documentation	■	■	■	■	■

22. Security Testing Lab

Product: Intentionally vulnerable test application.

Recommended stack: OWASP-style test cases, authentication/authorization, injection awareness and CI checks.

Project objective

Build a believable product workflow and demonstrate how a quality engineer would prevent, detect, diagnose and communicate failures.

Functional areas

• Authentication and authorization

• Core business workflow

• Negative and boundary cases

• Data validation

• Error handling

• Audit/history where relevant

Testing layers

Layer	What to demonstrate
Manual	Requirements, scenarios, risk-based cases, exploratory testing
API	Positive/negative, schema, auth, chaining, boundary conditions
Database	CRUD validation, joins, integrity, reconciliation
UI	Locators, synchronization, workflows, cross-browser
Automation	Fixtures, POM/component layer, data, assertions
CI/CD	Smoke gate, regression, reports, artifacts

Portfolio deliverables

■ README with architecture and setup

■ Test strategy and risk matrix

■ Test cases / scenarios

■ Automation repository

■ API collection or client

■ SQL validation examples

■ CI pipeline

■ Sample report and screenshots

■ Known limitations and future improvements

22. Security Testing Lab — Automation Blueprint

Recommended architecture: tests → workflows/page objects/API clients → fixtures/utilities → configuration/test data → reporting.

Suggested repository

tests/ UI/API/DB tests
pages/ or components/ UI abstraction
clients/ API clients
data/ controlled test data
utils/ reusable helpers
config/ environment settings
reports/ generated evidence
.github/ or Jenkinsfile CI pipeline

Definition of done

■ Tests run locally from a clean environment.

■ Tests can run in CI.

■ Failures provide useful evidence.

■ Test data does not create uncontrolled collisions.

■ README explains setup and architecture.

■ At least one meaningful quality metric is reported.

22. Security Testing Lab — Interview Defense Sheet

Questions an interviewer may ask

• Why did you choose this project?

• What were the highest-risk areas?

• What did you automate first and why?

• Why did you choose your framework?

• How did you manage test data?

• How did you handle flaky tests?

• How did you validate backend data?

• How does your CI pipeline work?

• What would you improve if this were production?

• What was the hardest problem you solved?

Your answer structure

Problem → Risk → Decision → Implementation → Evidence → Result → Trade-off → Next improvement

Portfolio score

Area	1	2	3	4	5
Testing depth	■	■	■	■	■
Automation quality	■	■	■	■	■
API/DB coverage	■	■	■	■	■
CI/CD	■	■	■	■	■
Documentation	■	■	■	■	■

23. Mobile Commerce App

Product: Product browsing, cart, checkout and account features.

Recommended stack: Appium, device coverage, API validation and mobile-specific scenarios.

Project objective

Build a believable product workflow and demonstrate how a quality engineer would prevent, detect, diagnose and communicate failures.

Functional areas

• Authentication and authorization

• Core business workflow

• Negative and boundary cases

• Data validation

• Error handling

• Audit/history where relevant

Testing layers

Layer	What to demonstrate
Manual	Requirements, scenarios, risk-based cases, exploratory testing
API	Positive/negative, schema, auth, chaining, boundary conditions
Database	CRUD validation, joins, integrity, reconciliation
UI	Locators, synchronization, workflows, cross-browser
Automation	Fixtures, POM/component layer, data, assertions
CI/CD	Smoke gate, regression, reports, artifacts

Portfolio deliverables

■ README with architecture and setup

■ Test strategy and risk matrix

■ Test cases / scenarios

■ Automation repository

■ API collection or client

■ SQL validation examples

■ CI pipeline

■ Sample report and screenshots

■ Known limitations and future improvements

23. Mobile Commerce App — Automation Blueprint

Recommended architecture: tests → workflows/page objects/API clients → fixtures/utilities → configuration/test data → reporting.

Suggested repository

tests/ UI/API/DB tests
pages/ or components/ UI abstraction
clients/ API clients
data/ controlled test data
utils/ reusable helpers
config/ environment settings
reports/ generated evidence
.github/ or Jenkinsfile CI pipeline

Definition of done

■ Tests run locally from a clean environment.

■ Tests can run in CI.

■ Failures provide useful evidence.

■ Test data does not create uncontrolled collisions.

■ README explains setup and architecture.

■ At least one meaningful quality metric is reported.

23. Mobile Commerce App — Interview Defense Sheet

Questions an interviewer may ask

• Why did you choose this project?

• What were the highest-risk areas?

• What did you automate first and why?

• Why did you choose your framework?

• How did you manage test data?

• How did you handle flaky tests?

• How did you validate backend data?

• How does your CI pipeline work?

• What would you improve if this were production?

• What was the hardest problem you solved?

Your answer structure

Problem → Risk → Decision → Implementation → Evidence → Result → Trade-off → Next improvement

Portfolio score

Area	1	2	3	4	5
Testing depth	■	■	■	■	■
Automation quality	■	■	■	■	■
API/DB coverage	■	■	■	■	■
CI/CD	■	■	■	■	■
Documentation	■	■	■	■	■

24. CI/CD Quality Gate Project

Product: Automated suite integrated into a deployment pipeline.

Recommended stack: Smoke/regression gates, artifacts, parallel tests, reports and failure triage.

Project objective

Build a believable product workflow and demonstrate how a quality engineer would prevent, detect, diagnose and communicate failures.

Functional areas

• Authentication and authorization

• Core business workflow

• Negative and boundary cases

• Data validation

• Error handling

• Audit/history where relevant

Testing layers

Layer	What to demonstrate
Manual	Requirements, scenarios, risk-based cases, exploratory testing
API	Positive/negative, schema, auth, chaining, boundary conditions
Database	CRUD validation, joins, integrity, reconciliation
UI	Locators, synchronization, workflows, cross-browser
Automation	Fixtures, POM/component layer, data, assertions
CI/CD	Smoke gate, regression, reports, artifacts

Portfolio deliverables

■ README with architecture and setup

■ Test strategy and risk matrix

■ Test cases / scenarios

■ Automation repository

■ API collection or client

■ SQL validation examples

■ CI pipeline

■ Sample report and screenshots

■ Known limitations and future improvements

24. CI/CD Quality Gate Project — Automation Blueprint

Recommended architecture: tests → workflows/page objects/API clients → fixtures/utilities → configuration/test data → reporting.

Suggested repository

tests/ UI/API/DB tests
pages/ or components/ UI abstraction
clients/ API clients
data/ controlled test data
utils/ reusable helpers
config/ environment settings
reports/ generated evidence
.github/ or Jenkinsfile CI pipeline

Definition of done

■ Tests run locally from a clean environment.

■ Tests can run in CI.

■ Failures provide useful evidence.

■ Test data does not create uncontrolled collisions.

■ README explains setup and architecture.

■ At least one meaningful quality metric is reported.

24. CI/CD Quality Gate Project — Interview Defense Sheet

Questions an interviewer may ask

• Why did you choose this project?

• What were the highest-risk areas?

• What did you automate first and why?

• Why did you choose your framework?

• How did you manage test data?

• How did you handle flaky tests?

• How did you validate backend data?

• How does your CI pipeline work?

• What would you improve if this were production?

• What was the hardest problem you solved?

Your answer structure

Problem → Risk → Decision → Implementation → Evidence → Result → Trade-off → Next improvement

Portfolio score

Area	1	2	3	4	5
Testing depth	■	■	■	■	■
Automation quality	■	■	■	■	■
API/DB coverage	■	■	■	■	■
CI/CD	■	■	■	■	■
Documentation	■	■	■	■	■

How to Choose Your Flagship Project

> Prefer one realistic project with multiple testing layers over ten toy repositories.

Action worksheet

Item	Plan / evidence
Business problem	
Core workflow	
Highest risks	
Test layers	
Automation scope	
CI/CD	
Evidence link / artifact	
Interview story	

Review questions

- Can another engineer run it?
- Can I explain every architectural choice?
- Does it demonstrate risk-based testing?
- Does it produce failure evidence?
- Would I be comfortable discussing it for 15 minutes in an interview?

Portfolio Architecture

Show how test code, application boundaries, data and CI fit together.

Action worksheet

Item	Plan / evidence
Business problem	
Core workflow	
Highest risks	
Test layers	
Automation scope	
CI/CD	
Evidence link / artifact	
Interview story	

Review questions

- Can another engineer run it?
- Can I explain every architectural choice?
- Does it demonstrate risk-based testing?
- Does it produce failure evidence?
- Would I be comfortable discussing it for 15 minutes in an interview?

GitHub README Template

Problem, product scope, architecture, stack, setup, tests, reports, CI, limitations and roadmap.

Action worksheet

Item	Plan / evidence
Business problem	
Core workflow	
Highest risks	
Test layers	
Automation scope	
CI/CD	
Evidence link / artifact	
Interview story	

Review questions

- Can another engineer run it?
- Can I explain every architectural choice?
- Does it demonstrate risk-based testing?
- Does it produce failure evidence?
- Would I be comfortable discussing it for 15 minutes in an interview?

Test Strategy Template

Scope, risks, test levels, environments, data, entry/exit criteria, metrics and release decision.

Action worksheet

Item	Plan / evidence
Business problem	
Core workflow	
Highest risks	
Test layers	
Automation scope	
CI/CD	
Evidence link / artifact	
Interview story	

Review questions

- Can another engineer run it?
- Can I explain every architectural choice?
- Does it demonstrate risk-based testing?
- Does it produce failure evidence?
- Would I be comfortable discussing it for 15 minutes in an interview?

Risk Matrix

Probability × impact × detectability; prioritize high-risk business flows.

Action worksheet

Item	Plan / evidence
Business problem	
Core workflow	
Highest risks	
Test layers	
Automation scope	
CI/CD	
Evidence link / artifact	
Interview story	

Review questions

- Can another engineer run it?
- Can I explain every architectural choice?
- Does it demonstrate risk-based testing?
- Does it produce failure evidence?
- Would I be comfortable discussing it for 15 minutes in an interview?

API Automation Project Blueprint

Clients, fixtures, schemas, data builders, auth and reporting.

Action worksheet

Item	Plan / evidence
Business problem	
Core workflow	
Highest risks	
Test layers	
Automation scope	
CI/CD	
Evidence link / artifact	
Interview story	

Review questions

- Can another engineer run it?
- Can I explain every architectural choice?
- Does it demonstrate risk-based testing?
- Does it produce failure evidence?
- Would I be comfortable discussing it for 15 minutes in an interview?

Playwright Project Blueprint

Locators, page/components, fixtures, storage state, tracing and parallelism.

Action worksheet

Item	Plan / evidence
Business problem	
Core workflow	
Highest risks	
Test layers	
Automation scope	
CI/CD	
Evidence link / artifact	
Interview story	

Review questions

- Can another engineer run it?
- Can I explain every architectural choice?
- Does it demonstrate risk-based testing?
- Does it produce failure evidence?
- Would I be comfortable discussing it for 15 minutes in an interview?

Selenium Project Blueprint

POM, synchronization, driver lifecycle, Grid and CI.

Action worksheet

Item	Plan / evidence
Business problem	
Core workflow	
Highest risks	
Test layers	
Automation scope	
CI/CD	
Evidence link / artifact	
Interview story	

Review questions

- Can another engineer run it?
- Can I explain every architectural choice?
- Does it demonstrate risk-based testing?
- Does it produce failure evidence?
- Would I be comfortable discussing it for 15 minutes in an interview?

PyTest Project Blueprint

Fixtures, parameterization, markers, hooks, plugins and reporting.

Action worksheet

Item	Plan / evidence
Business problem	
Core workflow	
Highest risks	
Test layers	
Automation scope	
CI/CD	
Evidence link / artifact	
Interview story	

Review questions

- Can another engineer run it?
- Can I explain every architectural choice?
- Does it demonstrate risk-based testing?
- Does it produce failure evidence?
- Would I be comfortable discussing it for 15 minutes in an interview?

Database Validation Blueprint

API action → DB query → expected state → reconciliation.

Action worksheet

Item	Plan / evidence
Business problem	
Core workflow	
Highest risks	
Test layers	
Automation scope	
CI/CD	
Evidence link / artifact	
Interview story	

Review questions

- Can another engineer run it?
- Can I explain every architectural choice?
- Does it demonstrate risk-based testing?
- Does it produce failure evidence?
- Would I be comfortable discussing it for 15 minutes in an interview?

CI/CD Portfolio Blueprint

PR smoke → merge regression → artifacts → quality gate.

Action worksheet

Item	Plan / evidence
Business problem	
Core workflow	
Highest risks	
Test layers	
Automation scope	
CI/CD	
Evidence link / artifact	
Interview story	

Review questions

- Can another engineer run it?
- Can I explain every architectural choice?
- Does it demonstrate risk-based testing?
- Does it produce failure evidence?
- Would I be comfortable discussing it for 15 minutes in an interview?

Dockerized Test Environment

Containerized dependencies and repeatable execution.

Action worksheet

Item	Plan / evidence
Business problem	
Core workflow	
Highest risks	
Test layers	
Automation scope	
CI/CD	
Evidence link / artifact	
Interview story	

Review questions

- Can another engineer run it?
- Can I explain every architectural choice?
- Does it demonstrate risk-based testing?
- Does it produce failure evidence?
- Would I be comfortable discussing it for 15 minutes in an interview?

Performance Project Blueprint

Workload model → baseline → load → stress → analyze → optimize.

Action worksheet

Item	Plan / evidence
Business problem	
Core workflow	
Highest risks	
Test layers	
Automation scope	
CI/CD	
Evidence link / artifact	
Interview story	

Review questions

- Can another engineer run it?
- Can I explain every architectural choice?
- Does it demonstrate risk-based testing?
- Does it produce failure evidence?
- Would I be comfortable discussing it for 15 minutes in an interview?

Security Project Blueprint

Threat-aware scenarios, auth, authorization, injection and secure test data.

Action worksheet

Item	Plan / evidence
Business problem	
Core workflow	
Highest risks	
Test layers	
Automation scope	
CI/CD	
Evidence link / artifact	
Interview story	

Review questions

- Can another engineer run it?
- Can I explain every architectural choice?
- Does it demonstrate risk-based testing?
- Does it produce failure evidence?
- Would I be comfortable discussing it for 15 minutes in an interview?

AI Testing Project Blueprint

Golden datasets, evaluation criteria, retrieval quality, groundedness and safety.

Action worksheet

Item	Plan / evidence
Business problem	
Core workflow	
Highest risks	
Test layers	
Automation scope	
CI/CD	
Evidence link / artifact	
Interview story	

Review questions

- Can another engineer run it?
- Can I explain every architectural choice?
- Does it demonstrate risk-based testing?
- Does it produce failure evidence?
- Would I be comfortable discussing it for 15 minutes in an interview?

Mobile Project Blueprint

Device matrix, permissions, network conditions, Appium and API support.

Action worksheet

Item	Plan / evidence
Business problem	
Core workflow	
Highest risks	
Test layers	
Automation scope	
CI/CD	
Evidence link / artifact	
Interview story	

Review questions

- Can another engineer run it?
- Can I explain every architectural choice?
- Does it demonstrate risk-based testing?
- Does it produce failure evidence?
- Would I be comfortable discussing it for 15 minutes in an interview?

Observability Portfolio Blueprint

Logs, metrics, traces, correlation IDs and failure evidence.

Action worksheet

Item	Plan / evidence
Business problem	
Core workflow	
Highest risks	
Test layers	
Automation scope	
CI/CD	
Evidence link / artifact	
Interview story	

Review questions

- Can another engineer run it?
- Can I explain every architectural choice?
- Does it demonstrate risk-based testing?
- Does it produce failure evidence?
- Would I be comfortable discussing it for 15 minutes in an interview?

Project Metrics That Impress

Coverage, pass rate, flaky rate, runtime, defect escape, pipeline duration and reliability.

Action worksheet

Item	Plan / evidence
Business problem	
Core workflow	
Highest risks	
Test layers	
Automation scope	
CI/CD	
Evidence link / artifact	
Interview story	

Review questions

- Can another engineer run it?
- Can I explain every architectural choice?
- Does it demonstrate risk-based testing?
- Does it produce failure evidence?
- Would I be comfortable discussing it for 15 minutes in an interview?

How to Demonstrate Business Impact

Translate automation into faster feedback, risk reduction and release confidence.

Action worksheet

Item	Plan / evidence
Business problem	
Core workflow	
Highest risks	
Test layers	
Automation scope	
CI/CD	
Evidence link / artifact	
Interview story	

Review questions

- Can another engineer run it?
- Can I explain every architectural choice?
- Does it demonstrate risk-based testing?
- Does it produce failure evidence?
- Would I be comfortable discussing it for 15 minutes in an interview?

Common Portfolio Mistakes

Copy-paste frameworks, fake metrics, huge README without evidence, brittle selectors and no CI.

Action worksheet

Item	Plan / evidence
Business problem	
Core workflow	
Highest risks	
Test layers	
Automation scope	
CI/CD	
Evidence link / artifact	
Interview story	

Review questions

- Can another engineer run it?
- Can I explain every architectural choice?
- Does it demonstrate risk-based testing?
- Does it produce failure evidence?
- Would I be comfortable discussing it for 15 minutes in an interview?

30-Day Project Build Plan

Week 1 scope; Week 2 manual/API/DB; Week 3 automation; Week 4 CI/documentation.

Action worksheet

Item	Plan / evidence
Business problem	
Core workflow	
Highest risks	
Test layers	
Automation scope	
CI/CD	
Evidence link / artifact	
Interview story	

Review questions

- Can another engineer run it?
- Can I explain every architectural choice?
- Does it demonstrate risk-based testing?
- Does it produce failure evidence?
- Would I be comfortable discussing it for 15 minutes in an interview?

60-Day Advanced Project Plan

Add architecture, Docker, performance/security and production-style diagnostics.

Action worksheet

Item	Plan / evidence
Business problem	
Core workflow	
Highest risks	
Test layers	
Automation scope	
CI/CD	
Evidence link / artifact	
Interview story	

Review questions

- Can another engineer run it?
- Can I explain every architectural choice?
- Does it demonstrate risk-based testing?
- Does it produce failure evidence?
- Would I be comfortable discussing it for 15 minutes in an interview?

90-Day Flagship Portfolio Plan

Build, stabilize, document, publish, present and use in interviews.

Action worksheet

Item	Plan / evidence
Business problem	
Core workflow	
Highest risks	
Test layers	
Automation scope	
CI/CD	
Evidence link / artifact	
Interview story	

Review questions

- Can another engineer run it?
- Can I explain every architectural choice?
- Does it demonstrate risk-based testing?
- Does it produce failure evidence?
- Would I be comfortable discussing it for 15 minutes in an interview?

Final Portfolio Checklist

Every artifact needed before publishing the project.

Action worksheet

Item	Plan / evidence
Business problem	
Core workflow	
Highest risks	
Test layers	
Automation scope	
CI/CD	
Evidence link / artifact	
Interview story	

Review questions

- ■ Can another engineer run it?
- ■ Can I explain every architectural choice?
- ■ Does it demonstrate risk-based testing?
- ■ Does it produce failure evidence?
- ■ Would I be comfortable discussing it for 15 minutes in an interview?

Final Portfolio Strategy

ONE FLAGSHIP + TWO SPECIALIZED PROJECTS + STRONG DOCUMENTATION + CI/CD + INTERVIEW STORY

A strong QA/SDET portfolio should prove that you can think like a tester, automate like an engineer, investigate like a debugger and communicate like a quality owner.

LEARN → PLAN → TEST → AUTOMATE → INTEGRATE → MEASURE → DOCUMENT → PRESENT → DEFEND → IMPROVE

PROJECT PORTFOLIO ADDENDUM

Three final pages to turn project work into interview-ready evidence

BUILD ONE PROJECT THAT YOU CAN DEFEND FOR 20 MINUTES.

Flagship Project — Definition of Done

Area	Evidence
Product understanding	User roles, workflows, business rules and highest-risk paths
Manual testing	Strategy, scenarios, exploratory notes and defect examples
API	Positive/negative cases, schema validation, auth, chaining and boundary tests
Database	Queries, integrity checks and API-to-DB reconciliation
UI automation	Stable locators, synchronization, reusable components and assertions
Framework	Fixtures, configuration, test data, utilities and reporting
CI/CD	PR smoke, regression pipeline, artifacts and failure diagnostics
Engineering quality	Flaky-test control, runtime, maintainability and code review
Documentation	README, architecture diagram, setup and known limitations
Interview story	Problem → decision → implementation → result → trade-off

Before Publishing

■ Remove secrets and real credentials.

■ Add a clean setup command.

■ Add sample report screenshots.

■ Explain how test data is created/reset.

■ Show at least one CI run.

■ Document known limitations honestly.

■ Tag a stable release.

■ Be able to explain every important file.

GITHUB PORTFOLIO PRESENTATION

Make the Repository Recruiter-Friendly

README FIRST → ARCHITECTURE SECOND → TEST EVIDENCE THIRD → CI RESULT FOURTH

README section	What to show
1. Overview	What product is being tested and why it matters
2. Scope	Features and test boundaries
3. Tech stack	Python, PyTest, Playwright/Selenium, API, SQL, Docker, CI
4. Architecture	Simple diagram and explanation of layers
5. Test strategy	Risk, levels, environments and entry/exit criteria
6. How to run	One clear local command plus environment setup
7. Automation	What is automated and why
8. Reporting	Screenshots, traces, logs and artifacts
9. CI/CD	Pipeline stages and quality gates
10. Results	Useful metrics with honest definitions
11. Defects	A few representative defects and root causes
12. Roadmap	What you would improve next

Evidence That Matters

- A failed test with useful trace/log evidence.
- A SQL validation query tied to a business workflow.
- An API negative test that catches a real rule violation.
- A CI pipeline with a visible quality gate.
- A documented flaky-test fix.
- A short architecture explanation.

PROJECT → INTERVIEW CONVERSION

Turn the Project into Interview Answers

Do not say: “I made a Playwright framework.”
Say: “I identified the highest-risk workflows, designed the automation layers, isolated test data, integrated the suite into CI and added diagnostics so failures could be investigated quickly.”

Interviewer asks	Answer evidence
Why this project?	Business problem + learning objective
What did you automate?	Risk-based selection + examples
Why Playwright/Selenium?	Technical requirement + trade-off
How is framework designed?	Layers + responsibilities
How do you validate backend?	API action + SQL reconciliation
How do you handle flaky tests?	Root cause + prevention
How does CI work?	Trigger + stages + artifacts + gate
What was hardest?	Specific failure + debugging path
What did you improve?	Before/after metric or qualitative evidence
What would you change?	Honest technical debt + next design step

Final Rule

Every project should answer three questions: What did you test? How did you engineer the automation? What evidence proves you did it?

Use this addendum with the main Project Ideas guide before publishing any portfolio repository.